



SIMATS ENGINEERING
CO5-ASSESSMENT TOOL -3
Saveetha Institute of Medical and Technical Sciences
Chennai- 602105



Student Name: BANDI MOHITH MANOJ

Reg. No.: 192472018

Course Code: CSA0619

Slot: D

Course Name: DESIGN AND ANALYSIS OF ALGORITHMS

Course Faculty: Dr. Mary Harin Fernandez F

EVALUATION SCORE REPORT

EVALUATION SCORE

Criterion	Weightage	Level	Marks Obtained
Technical Knowledge	25	Level 4	24/25
Problem Solving	25	Level 4	22/25
Analytical Thinking	15	Level 4	14/15
Communication	15	Level 4	12/15
Confidence	10	Level 3	8/10
Response to Questions	10	Level 4	10/10
TOTAL	100	Level 4	90/100

FINAL SCORE: 90/100

OVERALL PERFORMANCE LEVEL: LEVEL 4 – EXCELLENT

Assignment: Algorithm Design and Problem Solving

Question 1: What is a pruning technique in Backtracking? Why is it useful?

Answer:

Pruning means stopping a particular branch when we already know that it cannot give a valid solution. Instead of checking that branch until the end, we reject it immediately. This saves time because the algorithm does not explore unnecessary possibilities. For example, in N-Queens, if two queens are already attacking each other, we can stop exploring that branch.

Question 2: What is the difference between recursion and backtracking?

Answer:

Recursion is a technique where a function calls itself to solve a smaller part of the same problem. Backtracking uses recursion, but it also makes choices and goes back when a choice does not work. So, I would say recursion is the basic mechanism, while backtracking is a problem-solving strategy built using recursion and decision-making.

Question 3: How does the Subset Sum problem use a State-Space Tree?

Answer:

In Subset Sum, for every number we basically have two choices: include it or exclude it. These choices form the branches of the State-Space Tree. Each level represents one number, and each node represents the current subset or sum. If the current sum becomes greater than the target, when all numbers are positive, we can prune that branch.

Question 4: Why is N-Queens considering a Backtracking problem?

Answer:

N-Queens is suitable for backtracking because we have to make a sequence of choices, one queen at a time. After placing a queen, we check whether it conflicts with the previously placed queens. If there is a conflict, we remove the queen and try another position. This process continues until we find a valid arrangement.

Question 5: What is meant by the optimal substructure property?

Answer:

Optimal substructure means that the optimal solution to a problem can be obtained from optimal solutions of its smaller subproblems. For example, in Dynamic Programming, we solve smaller problems first and use their results to build the final answer. This property helps us avoid solving the same subproblem repeatedly.

Question 6: Why can some algorithms not guarantee an optimal solution?

Answer:

Some algorithms make decisions based on limited information or only the current situation. A Greedy algorithm, for example, chooses what looks best at that moment. That choice may not always lead to the best overall solution. Heuristic algorithms also focus on finding a good solution quickly instead of guaranteeing the absolute best solution.

Question 7: What is the purpose of a bounding function in an optimization problem?

Answer:

A bounding function helps us estimate whether a particular branch can still produce a better solution. If the estimated result is already worse than the best solution we have found, we can stop exploring that branch. This is useful in Branch and Bound because it reduces the number of possibilities we need to check.

Question 8: Suppose a graph has 5 vertices, and you need to color it using 3 colors. How would you approach it?

Answer:

I would use backtracking. I would take one vertex at a time and try the three colors. Before assigning a color, I would check whether any adjacent vertex already has the same color. If the color is safe, I continue to the next vertex. If none of the colors work, I go back to the previous vertex and change its color.

Question 9: Why are NP-Complete problems important in algorithm design?

Answer:

NP-Complete problems are important because they are considered computationally difficult problems. We can verify a given solution quickly, but finding the solution can take a lot of time for large inputs. Problems like Subset Sum and some graph problems belong to this category.

Because of this difficulty, we often look for backtracking, approximation, heuristic, or other practical approaches.

Question 10: Imagine an industrial company needs to make thousands of decisions every day. Would you choose an exact algorithm or a faster approximate approach?

Answer:

I would first look at the size of the problem and how important the exact answer is. If the problem is very large and decisions have to be made quickly, I would prefer an approximation or heuristic approach. An exact algorithm can give the best answer, but it may take too much time for a large input. In an industrial system, getting a good solution quickly can sometimes be more useful than waiting for the perfect solution.